

Demo de disparadores en MySQL

El principal requisito para ejecutar los Triggers MySQL es tener privilegios MySQL SUPERUSER (administrador).

Primero, crearemos la base de datos y tabla para la que se establecerá el trigger:

```
/* Crear la base de datos */  
CREATE DATABASE IF NOT EXISTS demoTrigger;  
  
/* Usar la base de datos creada */  
USE demoTrigger;  
  
/* Crear tabla */  
CREATE TABLE IF NOT EXISTS people (age INT, name varchar(150));
```

Creación de un trigger BEFORE INSERT

A continuación definiremos el trigger. Se ejecutará antes de cada sentencia *INSERT* para la tabla *people* y en el caso de que introduzcamos una persona en la tabla *people* con una edad negativa, cambiará la edad a 0. **Si intentamos crear el trigger como *AFTER INSERT* dará un error: *Updating of NEW row is not allowed in after trigger***

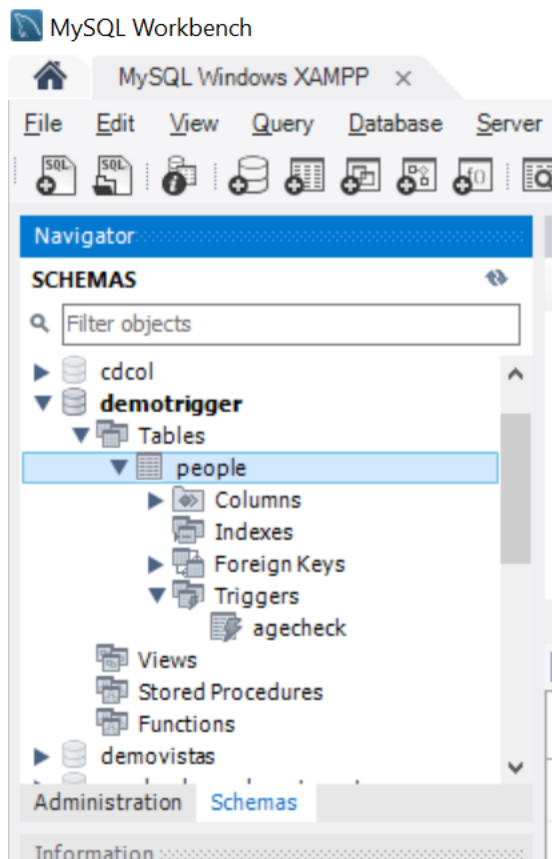
```
/* Especificamos un nuevo delimitador //  
para distinguir entre sentencias dentro  
de la creación del trigger y la finalización  
del trigger */  
delimiter //  
  
/* Creamos el trigger */  
CREATE TRIGGER agecheck  
BEFORE INSERT  
ON people FOR EACH ROW  
IF NEW.age < 0  
THEN SET NEW.age = 0; END IF;//  
  
/* Volvemos a especificar que el delimitador  
por defecto sea ; */  
delimiter ;
```

Insertaremos dos registros para comprobar la funcionalidad del trigger:
`INSERT INTO people VALUES (-20, 'Sid'), (30, 'Josh');`

Para terminar, comprobaremos el resultado, como se puede ver, a pesar de que Sid se ha intentado introducir con edad negativa, la edad es 0:
`SELECT * FROM people;`

age	name
0	Sid
30	Josh

Adicionalmente podemos verificar el trigger en MySQL Workbench:



```
/* Mostrar todos los triggers de la base de
datos demoTrigger y tabla people*/
SHOW TRIGGERS
FROM demoTrigger
LIKE 'people';
```

```
/* Mostrar todos los triggers de la tabla
people de nombre agecheck*/
SHOW TRIGGERS
FROM people
LIKE 'agecheck';
```

# Trigger	Event	Table	Statement	Timing	...
'agecheck'	'INSERT'	'people'	'IF NEW.age < 0 \nTHEN SET NEW.age = 0; END IF'	'BEFORE'	...

Creación de un trigger AFTER INSERT

```
DROP TABLE IF EXISTS members;
```

```
CREATE TABLE members (  
  id INT AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255),  
  birthDate DATE,  
  PRIMARY KEY (id)  
);
```

```
DROP TABLE IF EXISTS reminders;
```

```
CREATE TABLE reminders (  
  id INT AUTO_INCREMENT,  
  memberId INT,  
  message VARCHAR(255) NOT NULL,  
  PRIMARY KEY (id , memberId)  
);
```

La siguiente instrucción crea un trigger AFTER INSERT que inserta un recordatorio en la tabla *reminders* si la fecha de nacimiento del miembro en la tabla *members* es NULL. El identificador insertado en la tabla *reminders* corresponde al campo *id* de *members*:

```
DELIMITER $$
```

```
CREATE TRIGGER after_members_insert  
AFTER INSERT  
ON members FOR EACH ROW  
BEGIN  
  IF NEW.birthDate IS NULL THEN  
    INSERT INTO reminders(memberId, message)  
    VALUES(new.id, CONCAT('Hi ', NEW.name, ', please update your date of birth.'));  
  END IF;  
END$$
```

```
DELIMITER ;
```

Lo probamos, insertando dos filas en la tabla *members*:

```
INSERT INTO members(name, email, birthDate)  
VALUES  
  ('John Doe', 'john.doe@example.com', NULL),  
  ('Jane Doe', 'jane.doe@example.com', '2000-01-01');
```

/ Obtener los registros de la tabla members */*
*SELECT * FROM members;*

id	name	email	birthDate
1	John Doe	john.doe@example.com	NULL
2	Jane Doe	jane.doe@example.com	2000-01-01

/ Obtener los registros de la tabla reminders */*
*SELECT * FROM reminders;*

id	memberId	message
1	1	Hi John Doe, please u...

Insertamos dos filas en la tabla de *members*. Sin embargo, solo la primera fila tiene un valor de fecha de nacimiento NULL, por lo tanto, el disparador insertó solo una fila en la tabla de *reminders*.

Creación de un trigger BEFORE UPDATE:

DROP TABLE IF EXISTS sales;

```
CREATE TABLE sales (  
  id INT AUTO_INCREMENT,  
  product VARCHAR(100) NOT NULL,  
  quantity INT NOT NULL DEFAULT 0,  
  fiscalYear SMALLINT NOT NULL,  
  fiscalMonth TINYINT NOT NULL,  
  CHECK(fiscalMonth >= 1 AND fiscalMonth <= 12),  
  CHECK(fiscalYear BETWEEN 2000 and 2050),  
  CHECK (quantity >=0),  
  UNIQUE(product, fiscalYear, fiscalMonth),  
  PRIMARY KEY(id)  
);
```

```
INSERT INTO sales(product, quantity, fiscalYear, fiscalMonth)  
VALUES  
  ('2003 Harley-Davidson Eagle Drag Bike',120, 2020,1),  
  ('1969 Corvair Monza', 150,2020,1),  
  ('1970 Plymouth Hemi Cuda', 200,2020,1);
```

```
/* Verificar la inserción de datos */  
SELECT * FROM sales;
```

	id	product	quantity	fiscalYear	fiscalMonth
▶	1	2003 Harley-Davidson Eagle Drag Bike	120	2020	1
	2	1969 Corvair Monza	150	2020	1
	3	1970 Plymouth Hemi Cuda	200	2020	1

/ Se crea un trigger que se activa automáticamente antes de que se produzca un evento de UPDATE para cada fila de la tabla de sales.*

*Si actualiza el valor en la columna de quantity a un nuevo valor que es 3 veces mayor que el valor actual, el activador genera un error y detiene la actualización. */*
DELIMITER \$\$

```
CREATE TRIGGER before_sales_update  
BEFORE UPDATE  
ON sales FOR EACH ROW
```

```
BEGIN
  DECLARE errorMessage VARCHAR(255);
  SET errorMessage = CONCAT('The new quantity ',
    NEW.quantity,
    ' cannot be 3 times greater than the current quantity ',
    OLD.quantity);

  IF NEW.quantity > OLD.quantity * 3 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = errorMessage;
  END IF;
END $$

DELIMITER ;
```

Para probarlo, actualizamos *quantity* de la fila 1 a 150:

```
UPDATE sales
SET quantity = 150
WHERE id = 1;
```

Verificamos la actualización:
*SELECT * FROM sales;*

	id	product	quantity	fiscalYear	fiscalMonth
▶	1	2003 Harley-Davidson Eagle Drag Bike	150	2020	1
	2	1969 Corvair Monza	150	2020	1
	3	1970 Plymouth Hemi Cuda	200	2020	1

Ahora intentamos actualizar la cantidad de la fila con id 1 a 500 (más de 3 veces la cantidad actual):

```
UPDATE sales
SET quantity = 500
WHERE id = 1;
```

Salta el siguiente error:

Error Code: 1644. The new quantity 500 cannot be 3 times greater than the current quantity 150

Creación de un trigger AFTER UPDATE:

DROP TABLE IF EXISTS Sales;

```
CREATE TABLE Sales (
  id INT AUTO_INCREMENT,
  product VARCHAR(100) NOT NULL,
  quantity INT NOT NULL DEFAULT 0,
  fiscalYear SMALLINT NOT NULL,
  fiscalMonth TINYINT NOT NULL,
  CHECK(fiscalMonth >= 1 AND fiscalMonth <= 12),
  CHECK(fiscalYear BETWEEN 2000 and 2050),
  CHECK (quantity >=0),
  UNIQUE(product, fiscalYear, fiscalMonth),
  PRIMARY KEY(id)
);
```

```
INSERT INTO Sales(product, quantity, fiscalYear, fiscalMonth)
VALUES
('2001 Ferrari Enzo',140, 2021,1),
('1998 Chrysler Plymouth Prowler', 110,2021,1),
('1913 Ford Model T Speedster', 120,2021,1);
```

*SELECT * FROM Sales;*

	id	product	quantity	fiscalYear	fiscalMonth
▶	1	2001 Ferrari Enzo	140	2021	1
	2	1998 Chrysler Plymouth Prowler	110	2021	1
	3	1913 Ford Model T Speedster	120	2021	1

/ crear una tabla que almacene los cambios en la columna quantity de la tabla Sales */*
DROP TABLE IF EXISTS SalesChanges;

```
CREATE TABLE SalesChanges (
  id INT AUTO_INCREMENT PRIMARY KEY,
  salesId INT,
  beforeQuantity INT,
  afterQuantity INT,
  changedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```



```
/* Crear el trigger, si actualiza el valor en  
la columna de cantidad a un nuevo valor, el  
disparador inserta una nueva fila para registrar  
los cambios en la tabla SalesChanges. */  
DELIMITER $$
```

```
CREATE TRIGGER after_sales_update  
AFTER UPDATE  
ON sales FOR EACH ROW  
BEGIN  
    IF OLD.quantity <> NEW.quantity THEN  
        INSERT INTO SalesChanges(salesId,beforeQuantity, afterQuantity)  
        VALUES(old.id, OLD.quantity, NEW.quantity);  
    END IF;  
END$$  
  
DELIMITER ;
```

Lo probamos:

```
UPDATE Sales  
SET quantity = 350  
WHERE id = 1;
```

Verificamos que se ha almacenado el cambio:

```
SELECT * FROM SalesChanges;
```

	id	salesId	beforeQuantity	afterQuantity	changedAt
►	1	1	140	350	2019-09-07 13:24:58

Creación de un trigger BEFORE DELETE:

DROP TABLE IF EXISTS Salaries;

```
CREATE TABLE Salaries (
    employeeNumber INT PRIMARY KEY,
    validFrom DATE NOT NULL,
    amount DEC(12 , 2 ) NOT NULL DEFAULT 0
);
```

```
INSERT INTO salaries(employeeNumber,validFrom,amount)
VALUES
    (1002,'2000-01-01',50000),
    (1056,'2000-01-01',60000),
    (1076,'2000-01-01',70000);
```

DROP TABLE IF EXISTS SalaryArchives;

```
CREATE TABLE SalaryArchives (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employeeNumber INT,
    validFrom DATE NOT NULL,
    amount DEC(12 , 2 ) NOT NULL DEFAULT 0,
    deletedAt TIMESTAMP DEFAULT NOW()
);
```

/ Creamos un trigger que inserta una nueva fila
en la tabla SalaryArchives antes de que se elimine
una fila de la tabla Salaries */*

DELIMITER \$\$

```
CREATE TRIGGER before_salaries_delete
BEFORE DELETE
ON salaries FOR EACH ROW
BEGIN
    INSERT INTO SalaryArchives(employeeNumber,validFrom,amount)
    VALUES(OLD.employeeNumber,OLD.validFrom,OLD.amount);
END$$
```

DELIMITER ;

/ Lo probamos */*

/ Borramos un registro de la tabla Salaries */*

*DELETE FROM salaries
WHERE employeeNumber = 1002;*

/ Verificamos si se ha guardado */*

*SELECT * FROM SalaryArchives;*

	employeeNumber	validFrom	amount	deletedAt
►	1002	2000-01-01	50000.00	2019-09-07 21:00:18

Creación de un trigger AFTER DELETE:

```
/* Creación de un trigger AFTER DELETE */
DROP TABLE IF EXISTS Salaries;
```

```
CREATE TABLE Salaries (
    employeeNumber INT PRIMARY KEY,
    salary DECIMAL(10,2) NOT NULL DEFAULT 0
);
```

```
INSERT INTO Salaries(employeeNumber,salary)
VALUES
    (1002,5000),
    (1056,7000),
    (1076,8000);
```

```
DROP TABLE IF EXISTS SalaryBudgets;
```

```
/* Se crea otra tabla llamada SalaryBudgets que
almacena el total de salarios de la tabla Salaries: */
```

```
CREATE TABLE SalaryBudgets(
    total DECIMAL(15,2) NOT NULL
);
```

```
/* Se usa la función SUM() para obtener el salario
total de la tabla Salaries e insértelo en la tabla SalaryBudgets: */
```

```
INSERT INTO SalaryBudgets(total)
SELECT SUM(salary)
FROM Salaries;
```

```
/* Verificamos la tabla SalaryBudgets */
```

```
SELECT * FROM SalaryBudgets;
```

	total
▶	20000.00

```
/* Se crea un trigger que actualiza el salario
total en la tabla SalaryBudgets después de
eliminar una fila de la tabla Salaries: */
```

```
CREATE TRIGGER after_salaries_delete
AFTER DELETE
ON Salaries FOR EACH ROW
UPDATE SalaryBudgets
SET total = total - old.salary;
```

```
/* Probarlo, borramos una fila de la tabla Salaries */  
DELETE FROM Salaries  
WHERE employeeNumber = 1002;
```

```
/* Comprobamos la tabla SalaryBudgets*/  
SELECT * FROM SalaryBudgets;
```

	total
▶	15000.00

Borrar trigger:

```
/* Borrar trigger */  
SHOW TRIGGERS  
FROM demoTrigger;
```

```
/* Borramos uno, por ejemplo agecheck */  
DROP TRIGGER IF EXISTS agecheck;
```

```
/* Verificamos */  
SHOW TRIGGERS  
FROM demoTrigger;
```